The analog – digital conversion system and the debugger tool

1 Paper's purpose

By the end of this laboratory every student should have basic knowledge regarding the analog-digital subsystem inside the C8051F040 microcontroller and should be able to use its special function registers in assembler programs. Every student should also assimilate the IDE's debugging functions and should be able to use them in order to create error-proof programs.

2 Introduction

This paper presents:

2.1. The analog – digital conversion system,

2.2. Development software: Silicon Laboratories IDE – the debugger.

2.1 The analog – digital conversion system (ADC2)

The C8051F040 microcontroller is equipped with two analog-digital convertors: ADC0 and ADC2. ADC0 is a 12 bit convertor while ADC2 is an 8 bit convertor. This paper focuses on presenting the 8 bit convertor.

The ADC2 subsystem consists of an 8-channel, configurable analog multiplexer, a programmable gain amplifier, and a 500 ksps, 8-bit successive-approximation-register ADC with integrated track-and-hold. The multiplexer, programmable amplifier and data conversion modes are all configurable under software control via the Special Function Registers shown in Figure 2.1. The ADC2 subsystem is enabled and active only when the AD2EN bit in the ADC2 Control register (ADC2CN) is set to logic 1.

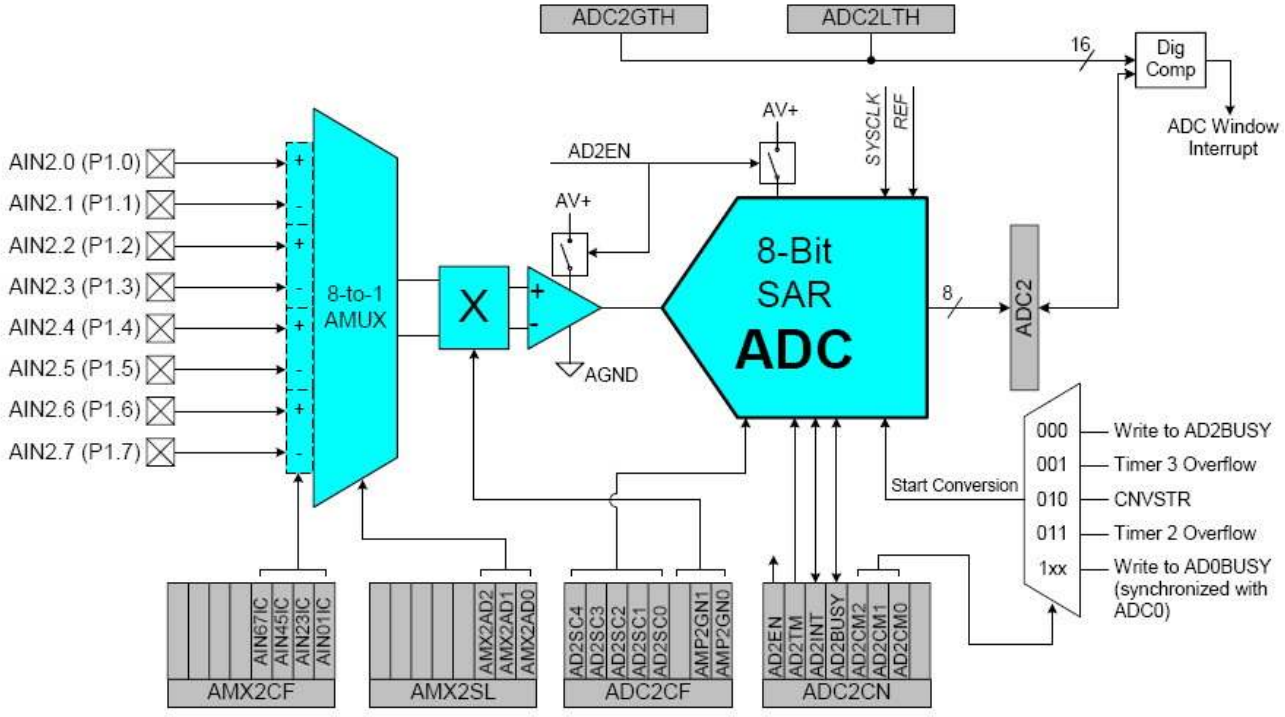


Fig. 2.1. ADC2 functional block diagram

2.1.1 Analog multiplexer and programmable gain amplifier

As one can notice from the diagram, the analog multiplexer's purpose is to select as an input into the system one of the eight analogical channels or the difference of two neighboring channels. The user can configure through software the input pins for the analog signal and its input mode: single-ended or differential. Two special function registers define the functionality for the analog multiplexer: AMX2CF (AMUX2 configuration register) and AMX2SL (AMUX2 channel select). The first four bits of AMX2CF configure the input mode (single ended or differential) for the four pairs of input pins. The first three bits of AMX2SL select an input pin or a pair of input pins as the multiplexer's output as shown in AMUXSelChart table. Refer to the microcontroller's datasheet (file C8051F040.pdf, page 95 an 96) for the definition of these two SFRs and the AMUXSelChart table.

Important Note: AIN2 pins also function as Port 1 I/O pins, and must be configured as analog inputs when used as ADC2 inputs. To configure an AIN2 pin for analog input, set to '0' the corresponding bit in register P1MDIN. Port 1 pins selected as analog inputs are skipped by the Digital I/O Crossbar.

The programmable gain amplifier performs a simple task as its name implies. It amplifies the ADC2 input analog signal by an amount determined by the states of the AMP2GN2-0 bits in the ADC2 Configuration register, ADC2CF. The amplifier can be software-programmed for gains of 0.5, 1, 2, or 4. Gain defaults to 0.5 on reset.

2.1.2 ADC2 modes of operation

ADC2 has a maximum conversion speed of 500 ksps. The ADC2 conversion clock (SAR2 clock) is a divided version of the system clock, determined by the AD2SC bits in the ADC2CF register (system clock divided by (AD2SC + 1) for $0 \leq AD2SC \leq 31$). The maximum ADC2 conversion clock is 7.5 MHz.

A conversion can be initiated in one of five ways, depending on the programmed states of the ADC2 Start of Conversion Mode bits (AD2STM2-AD2STM0) in ADC2CN. Conversions may be initiated by the following:

- Writing a '1' to the AD2BUSY bit of ADC2CN;
- A Timer 3 overflow (i.e. timed continuous conversions);
- A rising edge detected on the external ADC convert start signal, CNVSTR2 or CNVSTR0;
- A Timer 2 overflow (i.e. timed continuous conversions);
- Writing a '1' to the AD0BUSY of register ADC0CN (initiate conversion of ADC2 and ADC0 with a single software command.

This paper will explain the simplest "start of conversion" mode, which is writing a '1' to the AD2BUSY bit of ADC2CN. For more information regarding the other four conversion modes please refer to the microcontroller's datasheet (file C8051F040.pdf, page 92).

During conversion, the AD2BUSY bit is set to logic 1 and restored to 0 when conversion is complete. The falling edge of AD2BUSY triggers an interrupt (when enabled) and sets the interrupt flag in ADC2CN. Converted data is available in the ADC2 data word, ADC2. When a conversion is initiated by writing a '1' to AD2BUSY, it is recommended to poll AD2INT to determine when the conversion is complete. The recommended procedure is:

Step 1. Write a '0' to AD2INT;
Step 2. Write a '1' to AD2BUSY;
Step 3. Poll AD2INT for '1';
Step 4. Process ADC2 data.

For more information regarding the definitions of the ADC2CF, ADC2CN and ADC2 special function registers please refer to the microcontroller's datasheet (file C8051F040.pdf, page 97-99).

2.1.3 The Voltage Reference

The voltage reference circuit offers full flexibility in operating the ADC and DAC modules. Three voltage reference input pins allow each ADC and the two DACs (C8051F040/2 only) to reference an external voltage reference or the on-chip voltage reference output. ADC0 may also reference the DAC0 output internally, and ADC2 may reference the analog power supply voltage, via the VREF multiplexers shown in Figure 2.1.3.

The internal voltage reference circuit consists of a 1.2 V, temperature stable bandgap voltage reference generator and a gain-of-two output buffer amplifier. The internal reference may be routed via the VREF pin to

external system components or to the voltage reference input pins shown in Figure 2.1.3. Bypass capacitors of 0.1 µF and 4.7 µF are recommended from the VREF pin to AGND, as shown in Figure 9.1. See Table 9.1 for voltage reference specifications.

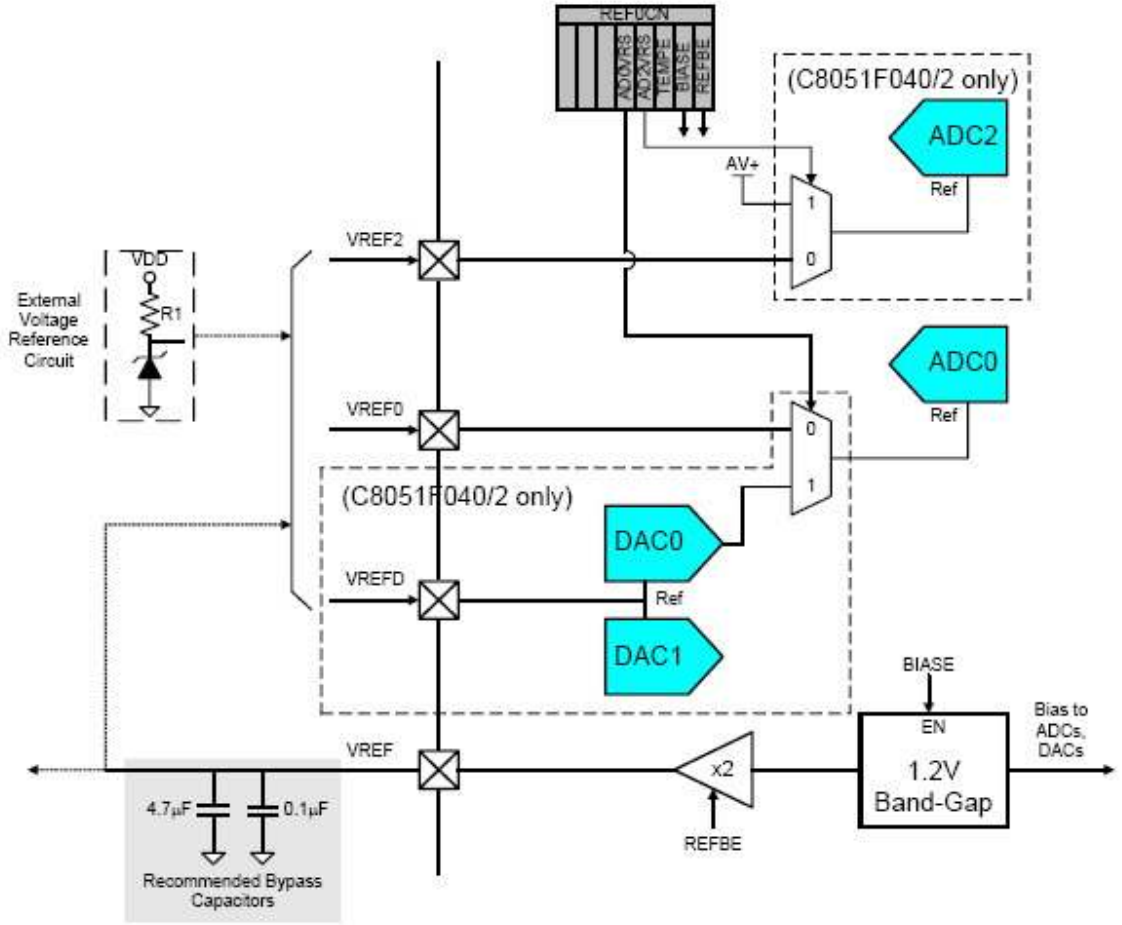


Fig. 2.1.3. Voltage Reference block diagram

The Reference Control Register, REF0CN enables/disables the internal reference generator and selects the reference inputs for ADC0 and ADC2. The BIASE bit in REF0CN enables the on-board reference generator while the REFBE bit enables the gain-of-two buffer amplifier which drives the VREF pin. If the internal bandgap is used as the reference voltage generator, BIASE and REFBE must both be set to logic 1. If the internal reference is not used, REFBE may be set to logic 0. Note that the BIASE bit must be set to logic 1 if either DAC or ADC is used, regardless of the voltage reference used. Bits AD0VRS and AD2VRS select the ADC0 and ADC2 voltage reference sources.

For more information regarding the definition of the REF0CN special function register please refer to the microcontroller's datasheet (file C8051F040.pdf, page 114).

2.2 Silicon Laboratories IDE – the debugger

Most of the microcontrollers offer a JTAG interface and the integrated circuitry logic to support a set of operations used in the testing and production processes. Throughout this interface an auxiliary independent system (usually a programmable system that can be a computer) is able to read and write the program memory (Flash), the internal and external data memory (RAM) and, consequently the general purpose registers and special function registers. This interface is also used to send execution commands, such as "step in", "step out", "execute to breakpoint", etc, to the microcontroller.

Silicon Laboratories IDE connects to this JTAG interface through an USB adapter and uses the interface for some of its particular functions: "Download Code", "Memory Fill", "Erase code space", "Upload Memory to File". It also keeps a permanent record of the values stored in the data memory (values that the user needs explicitly in the debug operation). Some of the IDE's functions mentioned above will be detailed in the following paragraphs.

2.2.1. Connecting the development board to the computer

As Silicon Laboratories IDE is not a simulator, we are required to connect the computer to the development board in order to exemplify the tools operation and the debugging functions. Given the same reason the execution of the programs can only be done directly on the attached hardware and not emulated through this integrated development environment.

Firstly you should setup the hardware following the steps given in Paper 1, section 2.1.5. Afterwards, you should start Silicon Laboratories IDE tool. Then use the `Options -> Connection Options` menu to configure the JTAG connection:

- select `Serial Adapter - USB Debug Adapter`.
- uncheck `Serial Adapter - Power target after disconnect`.
- select `Debug Interface - JTAG`.

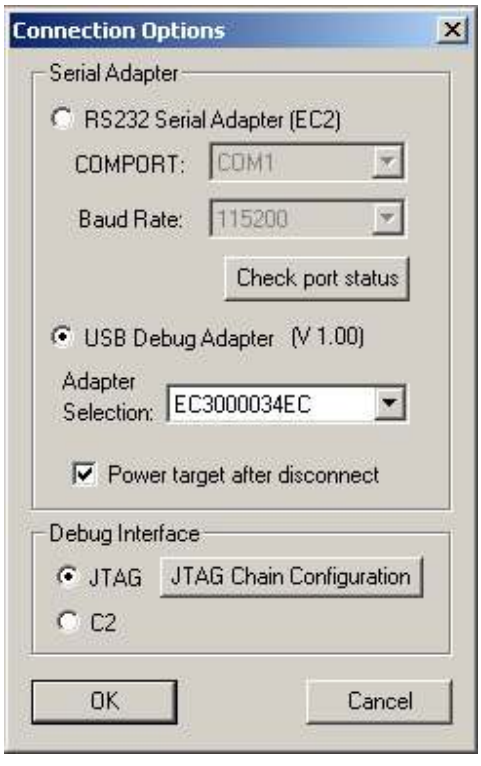


Fig. 2.2.1. Connection Options

Initiate the connection to the development board by pressing the `Connect` button in the toolbar or selecting the menu `Debug -> Connect`.

2.2.2. „Code/RAM memory fill" and „Erase code space" functions

In order to see the RAM data memory open the `Ram Viewer` window by pressing the `Ram Window` button in the toolbar or selecting the menu `View -> Debug Windows -> RAM`. The number columns in the center of this window represent the values stored in memory at the addresses shown in the left column, while the characters that appear in the right columns are the ASCII representations of the values stored in memory. The RAM memory can be easily modified by using the `RAM Memory Fill` function (selectable from the menu `Tools -> Memory Fill -> RAM`). Fill in the `Address Range Selection - Starting Address` and `Address Range Selection - Ending Address` fields, taking into account that the ending address has to be greater than the starting address. Type in a value in the `Pattern Selection - Byte for Fill` text field and press `Fill`. Follow the modifications in the `Ram Viewer` window. You have just modified the microcontroller's data memory.

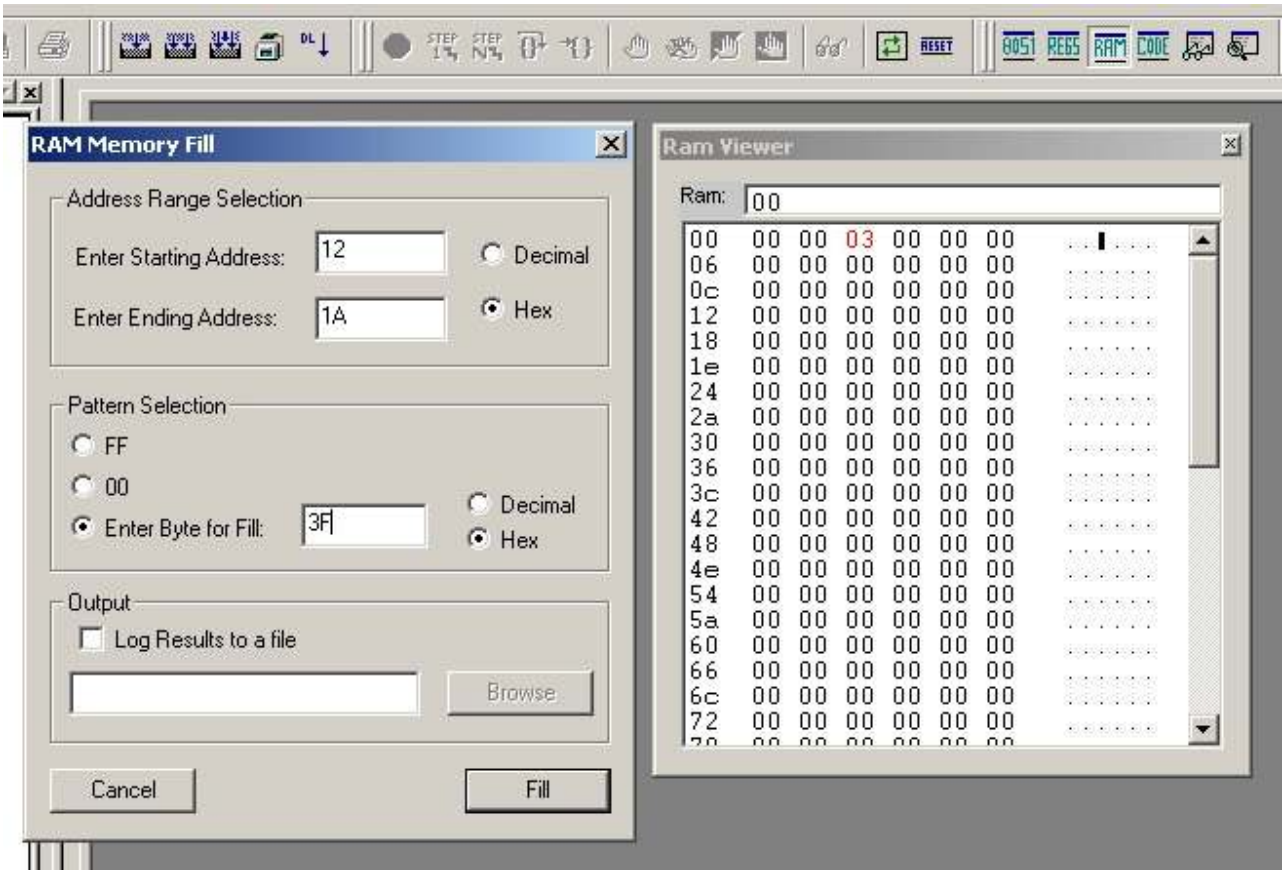


Fig. 2.2.2.a. RAM Memory Fill tool and Ram Viewer window

The microcontroller's program memory can be modified in the same manner. In order to visualize the program memory open the `Memory Viewer` window by pressing on the `Code Window` button in the toolbar or by selecting the menu `View -> Debug Windows -> Code Memory`. Analyze this window and note that the left column consists of addresses of the program memory. The columns in the center hold the values stored in the Flash memory, while the columns to the right contain the ASCII representations of the previous mentioned values. The Flash memory can be easily modified by using the `Code Memory Fill` function (selectable from the menu `Tools -> Memory Fill -> Code`). Fill in the `Address Range Selection - Starting Address` and `Address Range Selection - Ending Address` fields, taking into account that the ending address has to be greater than the starting address. Type in a value in the `Pattern Selection - Byte for Fill` text field and press `Fill`. Follow the modifications in the `Memory Viewer` window. You have just modified the microcontroller's program memory.

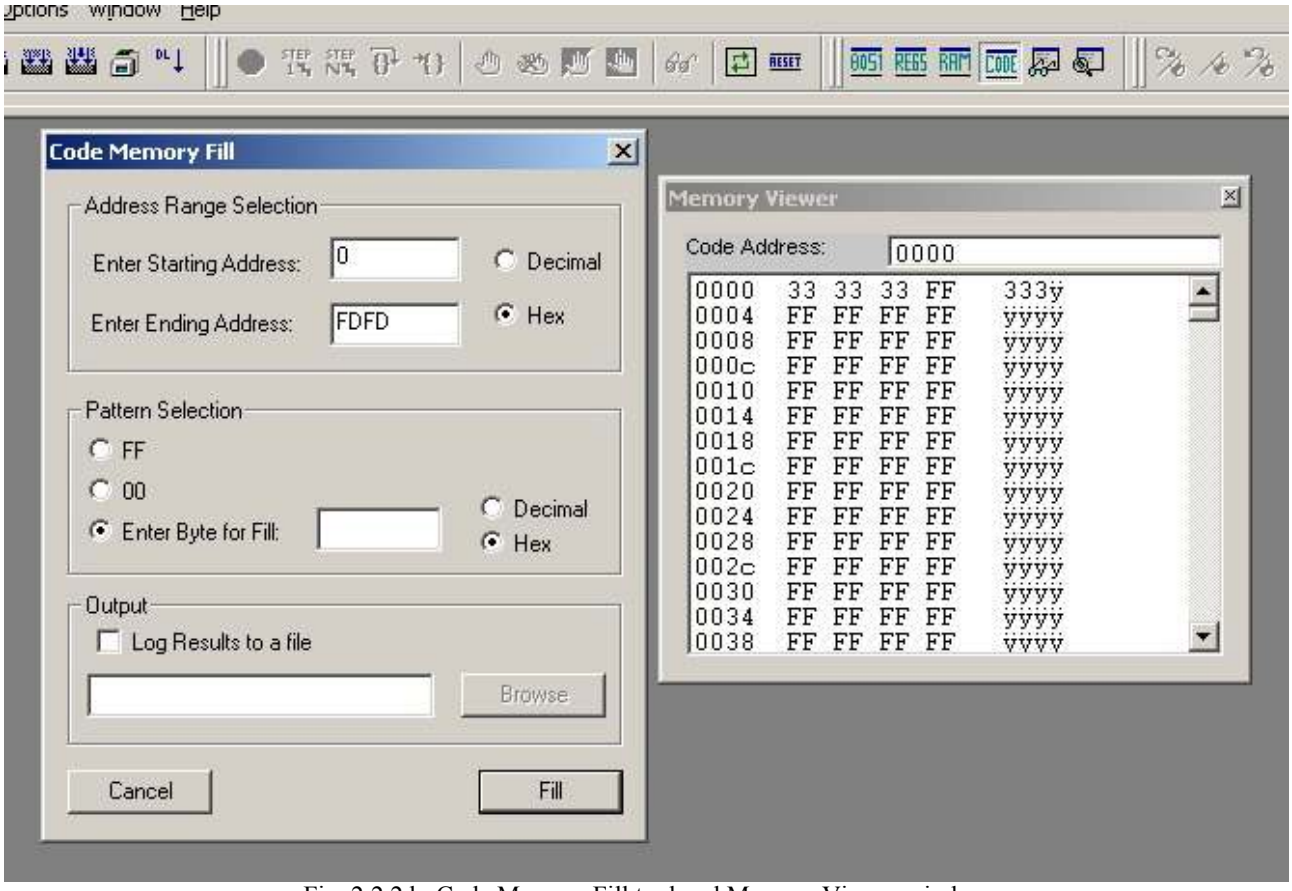


Fig. 2.2.2.b. Code Memory Fill tool and Memory Viewer window

The Flash memory in this microcontroller has the following particularities:

- it is organized in 512 bytes pages;
- a byte can only be overwritten through deletion: a bit that is set (has the logical value 1) can only be cleared (receives the logical value 0);
- a bit can be set (receive the logical value 1) only by setting all the bits in the page.

Silicon Laboratories IDE provides (besides the `Code Memory Fill` tool which modifies the program memory one byte at a time) another tool which resets the whole Flash memory (sets all the bits to the logical value of 1). Click on the `Code Window` button in the toolbar to visualize the program memory in the `Memory Viewer` window. Reset the Flash memory by selecting the menu `Tools -> Erase Code Space`. Note that all the values in the program memory were reset to the value 0xFF.

2.2.3. The debugger

Note: All the debugger's functions are only active when the computer is connected to the development board and a program is written in the microcontroller's code memory. The functions that are presented onward will be exemplified in section 4.

The program's execution can be started/stopped by pressing the `Go/Stop` button in the toolbar. The next button (`Step`), can be used for a step-by-step execution. The step-by-step execution can have a coarser granularity if the `Multiple Step` button is used instead. The multiple-step execution can be configured by selecting the menu `Multiple Step Configuration...`. The dialog enables you to set the number of instructions for every step and the refresh type between these steps (only the program counter PC or all the debug windows).

The next button, `Step over`, enables the user to skip execution the following instruction. There are two possibilities of executing the program up to a particular point in the source code and stop at the moment the program counter gets there. The `Run to cursor` button does exactly that without posing any restrictions

regarding the following execution of the program. The cursor shall be positioned on the desired code line and then the button can be pressed. The second possibility is more automated and involved the use of breakpoints. The IDE offers the possibility of setting breakpoints on any code line that stores an instruction by pressing the `Insert/Remove Breakpoints` button while the cursor is marking it. The effect is similar: the program gets executed up to that particular code line and the execution breaks before the selected instruction. Afterwards the breakpoint can be removed by pressing the same `Insert/Remove Breakpoints` button or simply disabled by pressing the `Enable/Disable Breakpoint` button. The breakpoint can then be enabled by pressing again the above mentioned button.

All the functions presented above can also be accessed through the `Debug` menu. These functions are extremely useful in case functional errors appear and the source code has to be debugged by viewing and real time modifying the program and data memory. The debug windows are only visible in an execution break and not while the program executes.

The most important information about an executing program are offered by these debug windows. These windows group the SFRs in several categories and enable the user to view and modify the values they store. These values are refreshed only in an execution break and not while the program executes. We will focus onward on several debug windows:

- 8051 Controller debug window
  - can be viewed by pressing the `8051 SFR's` button in the toolbar or by selecting the menu View -> `Debug Windows -> SFR's -> 8051 Controller/Misc`.
  - the most important registers it contains:
    - PC (program counter) – a register that stores the address of the current instruction.
    - SP (stack pointer) – a register that stores the address of the last element in the stack.
    - DPTR (data pointer) – a 16 bit register that is used as a pointer in the external data memory (64kB) or in the program memory (64kB).
    - PSW (program status word) – a register that stores the current state of the program. It stores info about the currently selected register bank, carry and overflow flags, etc. Please refer to the microcontroller's datasheet (file C8051F04xRev1_4.pdf, page 152) for more information on this register.
    - ACC (accumulator) – the main accumulator.
    - B – the secondary accumulator. This register is used in complex arithmetic operations in conjunction with the main accumulator.
    - SFRPAGE – a register that stores the index of the currently selected SFR page. Please refer to the microcontroller's datasheet (file C8051F04xRev1_4.pdf, page 142) for more information on this register.
- General purpose registers' debug window
  - can be viewed by pressing the `Register Window` button in the toolbar or by selecting the menu `View -> Debug Windows -> Registers`.
  - it contains the eight general purpose registers: R0, R1, ..., R7.
- Stack's debug window
  - can be viewed by selecting the menu `View -> Debug Windows -> Stack`.
  - contains:
    - SP (stack pointer) – a register that stores the address of the last element in the stack.
    - LIMIT – the upper limit for the stack.
    - COUNT – shows the number of bytes stored by the stack.
    - STACK CONTENTS – shows the content of the stack. The first column represents the address of the element, ne second column is the actual element, and the third column stores a description of the element).
- Ports' debug window
  - can be viewed by selecting the menu `View -> Debug Windows -> SFR's -> Ports`.
  - the most important registers it contains:
    - Px (x = 0..7) – the eight ports of the microcontroller.

- PxMDOUT (x = 0..7) – eight configuration registers corresponding to the eight ports. These registers configure the physical port (open-drain or push-pull).
- PxMDIN (x = 1..3) – three configuration registers corresponding to ports 1, 2 and 3. These registers determine the input mode (analog or digital) for these ports.

- The ADC2 (analog-digital convertor 2) debug window
  - can be viewed by selecting the menu `View -> Debug Windows -> SFR's -> ADC2`.
  - the most important registers it contains:
    - AMX2CF – configuration register for the analog multiplexer.
    - AMX2SL – selection register for the analog multiplexer.
    - ADC2CN – control register for the analog-digital convertor.
    - ADC2CF – configuration register for the analog-digital convertor.
    - ADC2 – output register for the analog-digital convertor.

Please refer to section 2 for more information regarding these registers.
All the debug windows mentioned above are presented in Fig. 2.2.3.

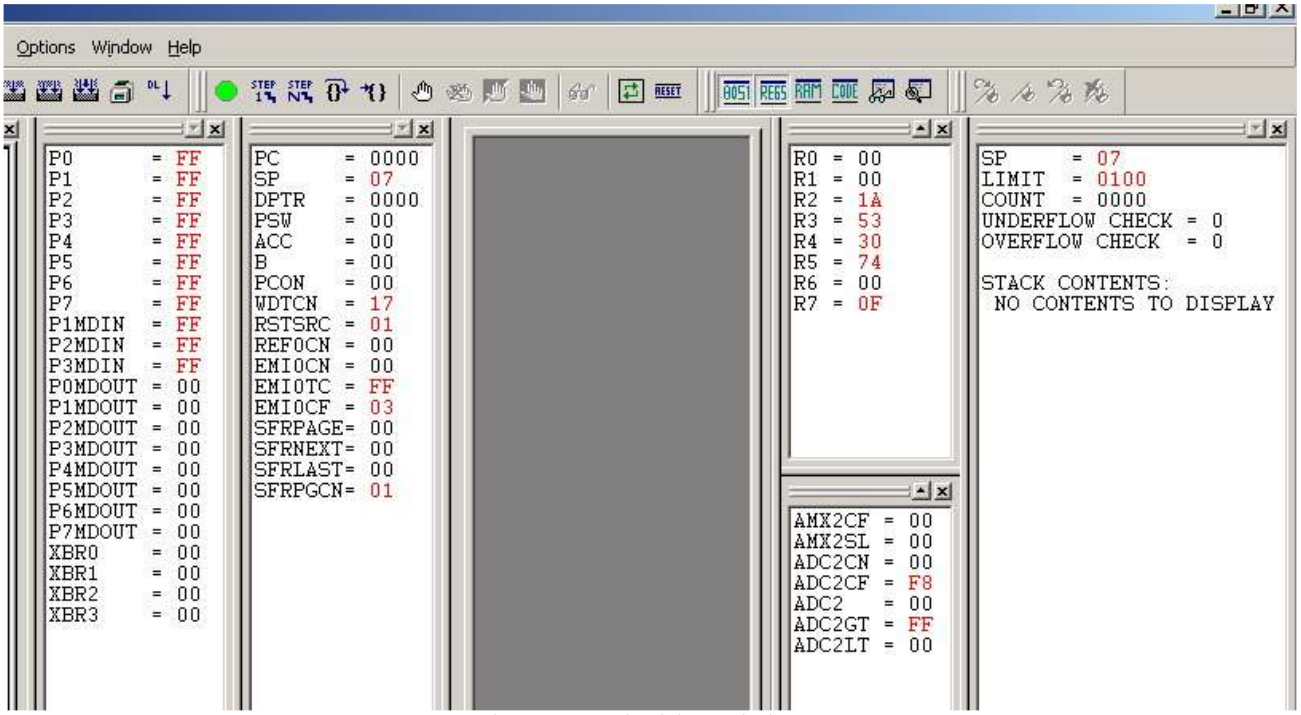


Fig. 2.2.3. Main debug windows

3 Application example

3.1 Requirements

Implement a system that measures and displays the light intensity in a room. Use a visual method for the displaying the output.

3.2 Proposed solution

3.2.1 Hardware module description

We will briefly present the implementation of a system that uses the C8051F040 development board and two additional hardware modules:

- input module
  - contains a solar cell that acts similar to a variable voltage generator (the output voltage is directly proportional to the light intensity).
  - it connects to the microcontroller through a pair of wires: one of them is the system's ground and the other routes the solar cell's output voltage.
  - the module's schematics is presented in Fig. 3.2.1.a.

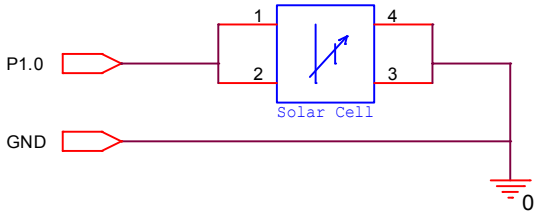


Fig. 3.2.1.a. Input module

- output module
  - contains, among other components that aren't subject to this paper, a bicolor red-green led that can be used as a display device.
  - it connects to the microcontroller through a 10 wire ribbon cable; only three wires out of the 10 are used by this application: the system's ground, the "red" anode of the bicolor led and the "green" anode of the bicolor led. Note: the bicolor led is a system that contains, inside the same led body, a red led and a green led with a shared cathode.
  - the schematic of the used part of the module is presented in Fig. 3.2.1.b.

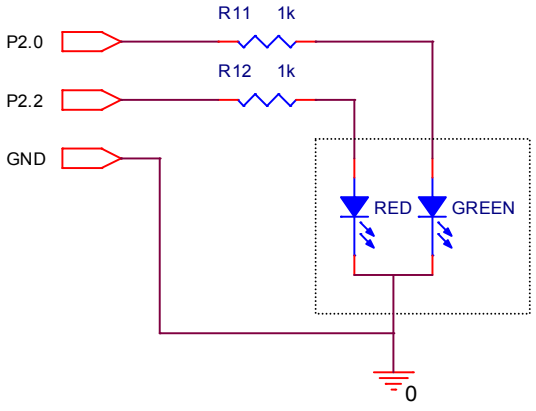


Fig. 3.2.1.b. Output module

As mentioned before the solar cell acts as a variable voltage (0 - 2.4V) generator. This voltage is routed to the P1.0 pin of the microcontroller and inside the analog multiplexer. The digital value obtained after the analog-digital conversion is

```
Cod = Vi * (K/Vref) * 256
```

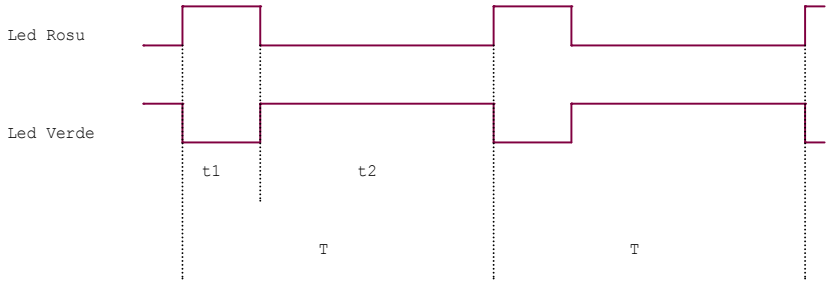
Where:

- `Cod` is an 8 bit number (between 0 - 255 decimal values)
- `Vi` is the input voltage
- `K` is the programmable amplifier gain (0.5, 1, 2, 4)
- `Vref` is the reference voltage

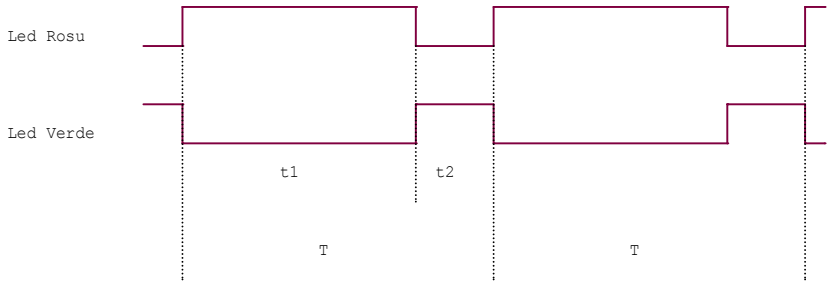
Note: the code obtain should be a number between 0 and 255, thus `Vi * (K/Vref)` should be lower than 1. Given this, the reference voltage `Vref` is chosen to be the internal voltage reference (2.4V) and the amplifier's gain `K` is set to 1. In order to obtain 2.4V as an internal reference voltage, the fixed gain amplifier has to be enabled (see section 2.1.3.). The internal reference voltage has to be hardware selected also by connecting a jumper between pins 5 and 6 of J22 connector.

A bicolor led is connected at P2.0 and P2.2 pins. This type of display device could have one of the following states at a given moment in time: completely off, red on, green on, or red and green on (an intermediary color is perceived). Still this application manages to display more than these four basic led states by modifying the fill factor for the step signals which are applied on the leds anodes.

As we've discussed in Paper 1, the human visual system is not able to perceive the blink of a led if it happens at a high frequency (higher than the critical frequency). This phenomenon is exploited by this application, the period of the signals applied to the leds being very short. In any of these periods the red led will be lit for a time interval noted t1, while the green led will be lit for a time interval noted t2. The ratio of these two values determines the color perceived by the human eye. Examples:



Example 1



Example 2

In Example 1, the color perceived by the human visual system is very close to green, with a pale red shade, while in the second example the red color is dominant. Note that it's necessary to keep the period T constant. This period is split into 16 equal time intervals, therefore the bicolor led can have 16 color levels between red and green (pure green will signify total darkness, while pure red will signify powerful light).

3.2.2 Algorithm description

Note that the algorithm and the source code follow the general structure presented in Paper 1.

The solution presented will be composed of several procedures: `main`, `baseDelay`, `init`, `lightTest`, `redOn`, `redOff`, `greenOn`, `greenOff`.

The algorithm consists of a fixed duration loop that infinitely iterates. The loop has three stages:

- the input voltage given by the solar cell is converted into a digital 8 bit value.
- the value obtained (a decimal number between 0 and 255) is divided by 16 and the resulted number (a decimal number between 0 and 15) represents the color level.

- the red led is lit for a time interval (t1) determined by the color level.
- the green led is lit for a time interval (t2) determined by the color level.

Note that the main loop has a fixed duration and therefore the sum of the time intervals (T=t1+t2) is also constant. This period is split into 16 equal time intervals.

The `lightTest` subroutine processes an analog-digital conversion. The obtained value (between 0 and 255) is divided by 16 resulting a number between 0 and 15, representing the number of short time intervals in which the red led will be on. This number is stored in the general purpose register R6. R7 stores the number of short time intervals in which the green led will be on (15-R6).

The `baseDelay` subroutine is a delay routine that uses R2 as a counter.

The main loop (`main`) calls the initialization subroutine and then enters the infinite loop labeled `start`. The `lightTest` subroutine which computes the values for R6 and R7 gets called here. The source code labeled `r` lights the red led only if necessary (the number of short time interval is not null). The source code labeled `redLoop` which holds the red led lit for a period of time equal to `R6*baseDelay` gets executed afterwards. Similarly the code labeled `g` and `greenLoop` holds the green led lit for a period of time equal to `R7*baseDelay` gets executed. In the end an unconditioned jump that's the program execution back to the beginning of the main loop.

The initialization subroutine

- disables the interrupt system
  - please see Paper 1 for details.
- disables the watchdog timer mechanism
  - please see Paper 1 for details.
- enables the crossbar
  - please see Paper 1 for details.
- configures port P1
  - the program configures the pin 0 of port P1 as analog input for the analog-digital convertor. This is done by modifying the value stored in P1MDIN.
  - refer to the microcontroller's datasheet (file C8051F04xRev1_4.pdf, page 219) for more info on the definition of P1MDIN special function register.
- configures port P2
  - the program uses pin 0 and pin 2 of port P2 to command the two leds. Therefore pins P2.0 and P2.2 have to be configured as digital output pins. The output mode will be push-pull. This is done by modifying the value stored in P2MDOUT.
  - note: in order to not disturb any other circuitry connected to the board through the ribbon cable all the port's pins are initially routed to ground.
  - refer to the microcontroller's datasheet (file C8051F04xRev1_4.pdf, page 218) for more info on the definition of P2MDOUT special function register.
- configures the voltage reference
  - the internal voltage reference is enabled by setting the BIASE and REFBE bits of the REF0CN register.
  - refer to the microcontroller's datasheet (file C8051F04xRev1_4.pdf, page 114) for more info on the definition of REF0CN special function register.
- configures the analog-digital convertor
  - P1.0 is selected as single-ended input in the analog multiplexer.
  - the convertor is configured to start a conversion whet the AD2BUSY gets set and the programmable gain amplifier is configured to have the gain 1.

The algorithm is also presented in Fig. 3.2.2.

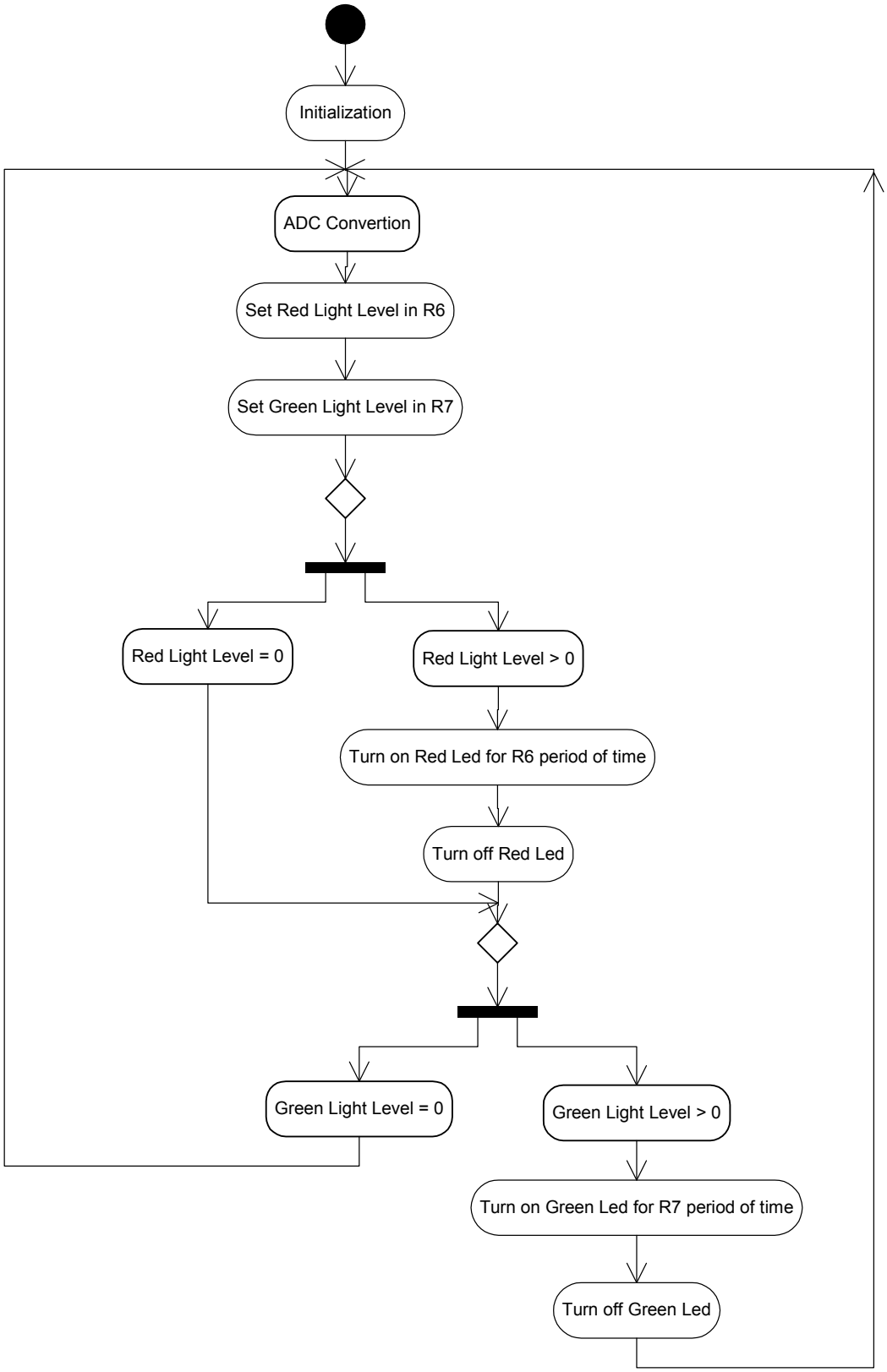


Fig. 3.2.2. RedGreenLed algorithm

4 Exercises

1. Create a new project using the file Z:\Lab8051\Laborator1\redGreenLed.asm and following the steps presented in Paper 1, Section 2.2.
2. Complete the initialization subroutine to disable the interrupts and the watchdog time and to enable the crossbar. Hint: follow the explanations given in Paper 1, Section 3.2.2.
3. Complete the on and off subroutines for the two leds. Hint: follow the explanations given in Paper 1, Section 3.2.2.
4. Execute the application by pressing the `Go` button in the IDE's toolbar and note the system behavior while the light intensity changes.
5. Stop the execution by pressing the `Stop` button and reset the system by pressing the `Reset` button. Note the blue marker placed on the left border of the edit window. Why does the program restart from that particular line of code when a reset occurs?

   Note that at this point the reset was software initiated through the IDE, but that there are many other reset sources as presented in Paper 1, Section 2.1.4.1.3.
6. Start a step by step execution as explained in Section 2.2.3.
7. Stop the stepping before the watchdog timer reset instruction block. Visualize the 8051 Controller debug window as instructed in Section 2.2.3. Note that the watchdog timer control register (WDTCN) resides in this debug window. Refer to the microcontroller's datasheet (file C8051F04xRev1_4.pdf, page 171) to determine if the watchdog timer is enabled or disabled. Continue the step by step execution and note the effect of these two instructions that are supposed to disable the watchdog timer.
8. Continue the step by step execution and visualize the ADC2 debug window as instructed in Section 2.2.3. Note the initial values of the configuration registers AMX2CF, AMX2SL, ADC2CF, ADC2CN and the values given in the initialization subroutine and compare them with the definition of these registers (file C8051F04xRev1_4.pdf, page 95, 97, 98) and the source code comments.
   - Which of these registers set the gain for the programmable gain amplifier?
   - What's the value set for the gain?
   - Which pin/pins is selected as input in the analog multiplexer? Is the input mode single-ended or differential? Which of the above registers set the input mode?
9. Continue the stepping up until the call of the `lightTest` subroutine. Note how the value in ADC2 updates after the ADC2BUSY bit is set and therefore starts a conversion.
10. Try to optimize the following block of instructions to get similar functionality in less clock cycles:

```
mov         B, #0x10;
div         AB
```

11. Once you've passed by the actual conversion visualize the general purpose registers debug window and the data memory window (RAM Viewer). Continue the step by step execution. Note that after the converted value is divided by 16, the resulted value is stored in R6 and R7. Note the correspondence between the general purpose registers and the data memory. What register bank is selected at this point (refer to the definition of PSW in file C8051F04xRev1_4.pdf, page 152)? Where is this register set mapped inside the data memory (refer to Paper 1, Section 2.1.4.2.4.)? Are the values in the memory and in the general purpose registers the same?
12. Continue the stepping up until the beginning of the code block labeled `r`. This code block keeps the red led lit for a number of `baseDelays` equal to the value stored in R6. Visualize the ports debug window. Continue the stepping and enter the `redOn` subroutine. Note the value in P2 just before the execution of the `setb RED` instruction and its value after the execution. Which of the bits have changed? Why? Do you notice anything on the display module?
13. Visualize the general purpose registers debug window and the 8051 Controller debug window. Continue the stepping and enter the `baseDelay` subroutine. What's the effect of the `djnz R2, $` instruction on the value stored in R2 and on the value stored in PC? Modify the value stored in R2 from the debug window in order to double the delay introduces by the `baseDelay` subroutine. Continue the stepping and enter the `baseDelay` procedure again. Is the modification previously applied on R2 still valid?
14. Reset the microcontroller by pressing the `Reset` button in the toolbar. Visualize the stack debug window as instructed in Section 2.2.3. What's the value stored in the stack pointer register? What does the stack

contain at this point? Visualize the data memory window and see if the values in the stack and the memory are correlated in any way.

Restart the stepping. What's the effect of the `ljmp main` instruction on the stack? Explain. Write down the PC's value before executing the `acall init` instruction. Execute the instruction. Has the stack changed? What about the data memory? Do you see the correlation? Explain the values stored in SP, COUNT and STACK CONTENTS using the information presented in Section 2.2.3. Why do we have the values 0x00 and 0x05 stored into the stack at this point? What's the size of the `acall init` instruction (see Paper 1, Section 2.1.4.1.4.)? What's the address of the following instruction in the main routine given the previously written down value of the PC? What storing convention was used to store this 16 bit number in the stack (little endian/big endian)?

15. Continue the stepping up until the `acall redOff` instruction. What happens with the stack as it gets executed? Write down the values of the last two bytes in the stack. What happens when the `ret` instruction gets executed? At what address did the program return in the calling routine?
16. Follow the steps:
    - Modify the `redOff` subroutine so that it contains an instruction (`push`) that inserts an element in the stack.
    - Save the project, assemble it, compile it and download it into the microcontroller as instructed in Paper 1, Section 2.2.
    - Use the "run to cursor" function to get right before the execution of the `acall redOff` instruction in the main routine. Note that the program was executed up to this point.
    - Write down the program counter's value, calculate the address of the following instruction and execute the instruction. Has the stack modified as predicted?
    - Execute the newly `push` instruction introduced in this subroutine and note the new value introduced in the stack.
    - Continue stepping up until the return instruction. What do you think would happen when you execute it? At which address will the program "return" to?
    - Execute the instruction and see the result. What happens if you continue stepping?

    Please note this totally inadequate behavior of execution. It happens when the stack is not managed in a safe manner. Inside a subroutine the programmer is responsible to pop out of the stack all the values he pushed in that block of code. For example, if you also introduce a `pop` instruction before the return is executed there would be no errors.

    A less noticeable error could occur if the data memory is directly modified by the program. As you have seen the stack is mapped in this data memory. If the RAM is modified in those areas where the stack resides the stack gets modified also. You've also seen that the general purpose registers are mapped inside the data memory, so consequently, the simple modification of a general purpose register could have disastrous effects on the stack. Try it!

    In order to avoid these memory management errors the stack pointer (and thus the stack) can be placed in an unused memory area. Of course this operation should be done only at the beginning of the program.
17. Modify the source code in order to have the initial functionality for the program.
18. Modify the program in order to obtain a complementary behavior: red led fully lit when the light is really low and green led fully lit when the light is powerful. Find the easiest way to do this.

5 Appendixes

5.1 RedGreenLed Application source code

```
$include (c8051f040.inc)                          ; Include register definition file.

;-----------------------------------------------------------------------------
; EQUATES
;-----------------------------------------------------------------------------

       RED             equ        P2.2
       GREEN           equ        P2.0

;-----------------------------------------------------------------------------
; RESET and INTERRUPT VECTORS
;-----------------------------------------------------------------------------

                       cseg       AT 0x0000           ; Reset Vector
                       ljmp       main                ; Locate a jump to the start of code
                                                      ;at the reset vector.
;-----------------------------------------------------------------------------
; MAIN PROGRAM CODE SEGMENT
;-----------------------------------------------------------------------------

mainCodeSeg            segment    CODE

                       rseg       mainCodeSeg         ; Switch to this code segment.
                       using      0                   ; Specify register bank for the following
                                                      ; program code.

main:                  acall      init                ; Initialization of all used SFR's
                       mov        SFRPAGE, #ADC2_PAGE ; Use SFRs on the ADC2 Page

start:                                                ; Starting the algorithm

                       acall      lightTest           ; Tests the intensity of the light

    r:                 mov        A, R6
                       jz         g
                       acall      redOn
    redLoop:           acall      baseDelay           ; Keeps the red LED lighted for
                       dec        A                   ; R6 base_delays
                       jnz        redLoop
                       acall      redOff              ; Red loop is over. Turn off the red LED

    g:                 mov        A, R7
                       jz         start
                       acall      greenOn
    greenLoop:         acall      baseDelay           ; Keeps the green LED lit for
                       dec        A                   ; R7 base_delays
                       jnz        greenLoop
                       acall      greenOff            ; Green loop is over. Turn off the green LED

                       sjmp       start               ; Start over again

;-----------------------------------------------------------------------------
; FUNCTION CODE
;-----------------------------------------------------------------------------

init:
                       ...                            ; Disable global interrupts
initWatchDogTimer:     ...                            ; Disable WatchDog Timer

initCrossbar:          ...                            ; Enable Crossbar

initIOPorts:           anl        P1MDIN, #0xFE       ; Configure P1.0 as analog input
                       orl        P2MDOUT, #0x05      ; Set P2.0 and P2.2 (GREEN/RED) as digital
                                                      ;output in push-pull mode.
                       mov        P2, #0x00           ; Route P2 to ground

initVREF:              mov        SFRPAGE, #0x00      ; Use SFRs on the 0x00 Page
                       mov        REF0CN, #0x03       ; ADC2 voltage reference from internal VREF
                                                      ; Enable Bias Generator

initADC2:              mov        SFRPAGE, #ADC2_PAGE       ; Use SFRs on the ADC2 Page
```

```
                    mov     AMX2CF, #0x00       ; Set AIN1.0 as single-ended input
                    mov     AMX2SL, #0x00       ; Select AIN1.0 as input channel for
                                                ;the analog multiplexer
                    mov     ADC2CF, #0xF9       ; Configure a*1 Gain
                    mov     ADC2CN, #0x80       ; Enable ADC2; Continuos tracking;
                    ret

baseDelay:          mov     R2, #0x0F
                    djnz    R2, $
                    ret

redOn:              ...                         ; Turn on the Red LED

redOff:             ...                         ; Turn off the Red LED

greenOn:            ...                         ; Turn on the Green LED

greenOff:           ...                         ; Turn off the Green LED


lightTest:          clr     AD2INT
                    setb    AD2BUSY             ; Force ADC2 to start a conversion
                    jnb     AD2INT, $           ; Wait for ADC2 to finish the conversion

                    mov     A, ADC2             ; Write the converted value in the accumulator
                    mov     B, #0x10            ; Divides the value by 16.
                    div     AB                  ; [0x00, 0xFF] -> [0x00, 0x0F]
                    mov     R6, A               ; R6 stores the resulted value
                    mov     A, #0x0F
                    clr     C
                    subb    A, R6
                    mov     R7, A               ; R7 = 0x0F - R6
                    ret

;------------------------------------------------------------------------------
; End of file.

END
```